Usage Metering

billing you can prove, quotas you can enforce

Watch the 4-min video

In one line: emit an immutable **usage event** with a unique ID for every billable action, write it to a durable log, **deduplicate and aggregate** per tenant, meter and period, check quotas against a fast counter, and send period totals to billing, so every number on an invoice can be traced back to events.

1 The disputed bill

Your AI platform charges per API call and per token. A large customer disputes the month: they count 800,000 calls, you billed 1.2 million. If your number came from a database counter incremented in the request path, you can't prove anything: counters lose updates when a write fails, double-count when a client retries, and keep no history. Billing needs a pipeline built like an accounting ledger: append-only facts, then derived totals you can recompute.

2 Usage events: facts, not counters

```
{ "event_id": "01J9Z6…",            // unique; set once, reused on retry
  "tenant_id": "acme",  "meter": "input_tokens",
  "quantity": 1834,     "unit": "token",
  "occurred_at": "2026-11-03T09:14:07Z",  // event time, not arrival time
  "dims": { "model": "large-v2", "api_key": "k_51", "request_id": "r-88…" } }
```

Figure 1. A usage event. Small, immutable, with an ID that survives retries and the dimensions you'll need to explain the bill.

Every billable action emits an event: tenant, meter (`api_calls`, `input_tokens`, `output_tokens`, `storage_gb_hours`), quantity, event time and a unique ID. Events go to a durable, replayable log such as Kafka (and are archived to object storage), not straight into a counter. The log is the source of truth: if an aggregation bug is found, you fix the job and recompute. Decide what is billable up front and write it into the contract: for example, count only successful calls, and count tokens actually processed.

3 Never bill a retry twice

Events get duplicated: a client retries a timed-out request, a producer re-sends after a lost ack, a consumer re-reads after a crash. So the ID must be **deterministic per billable action**, typically derived from the request ID or the client's `Idempotency-Key`, not generated fresh on each attempt. The pipeline then drops any ID it has already seen.

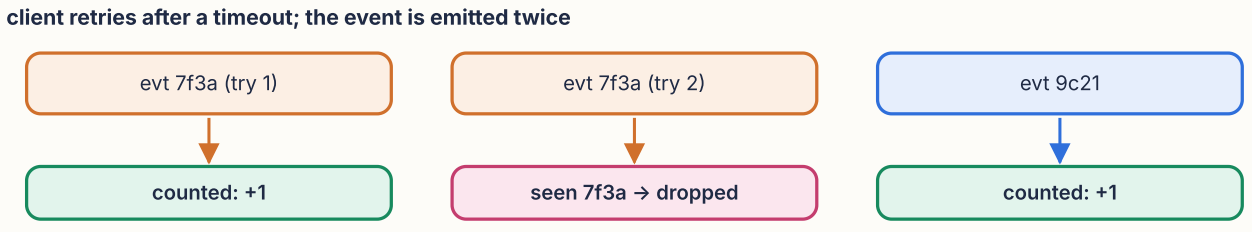


Figure 2. Deduplication by event ID: the retried request carries the same ID and is counted once.

Where to deduplicate: Kafka's idempotent producer and transactions give exactly-once processing *inside* Kafka (no duplicates from producer retries, atomic read-process-write), but they can't know two requests from a client are the same action. So keep the ID dedupe in the stream job (Flink keyed state of seen IDs with a TTL longer than any retry window, e.g. 7 days), and make the sink idempotent: upsert the total for `(tenant, meter, day)` rather than incrementing it.

4 Aggregate by event time, close periods carefully

A stream job rolls events up per tenant and meter: per minute for dashboards and quota checks, per day and per month for billing. Aggregate by **event time**, so a call at 23:59:58 on 31 October that arrives at 00:00:03 lands in October. Watermarks tell the job when a window is probably complete; allowed lateness accepts stragglers for a while. After a billing period is **closed and invoiced**, never rewrite it silently: late or corrected usage becomes an explicit adjustment line on the next invoice.

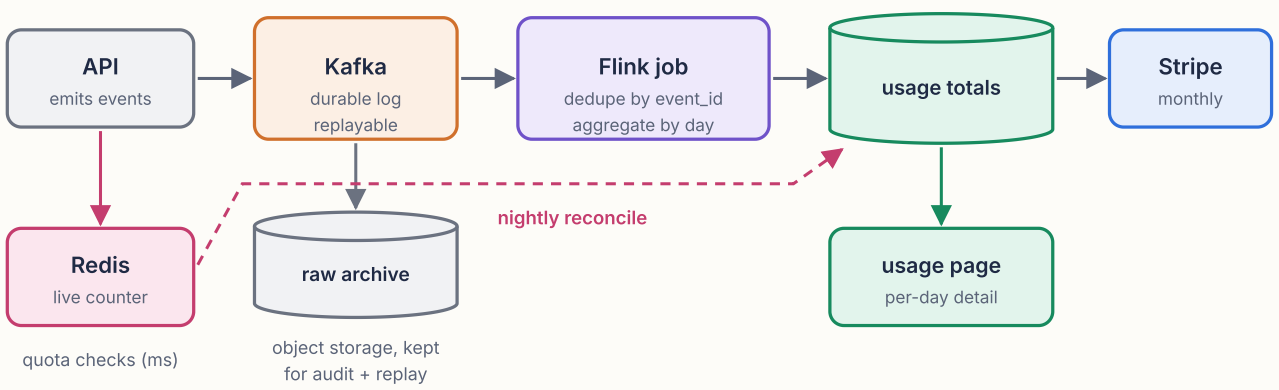


Figure 3. Metering pipeline. The slow, exact path feeds billing; the fast, approximate path feeds quota checks; reconciliation ties them together.

5 Quotas on the same numbers

A **soft limit** warns: at 80% of the plan the customer gets an email and a response header. A **hard limit** blocks (429 with a clear quota-exceeded error) or switches to **overage** pricing, depending on the plan. Quota checks run on every request and can't wait for the pipeline, so a fast counter tracks this period in near real time: `INCRBY usage:acme:input_tokens:2026-11 1834` in Redis, with the key expiring after the period. For tokens, you don't know the cost until the model answers, so **reserve** an estimate (e.g. max_tokens) before the call and settle the difference after.

	Fast counter (Redis)	Durable totals (pipeline)
Purpose	quota decisions, live usage bar	invoices, disputes, analytics
Latency	milliseconds	seconds to minutes
Accuracy	approximate: may miss or double a few	exact: deduplicated, replayable
If lost	rebuild from durable totals	recompute from the log / archive

Reconcile nightly: compare the counter with the durable totals and correct the counter. The counter is **advisory**; the durable totals are what you bill.

6 Billing integration

At period end, send each tenant's per-meter totals to the billing system (Stripe's usage-based billing, or open-source tools such as Lago or OpenMeter), which applies the plan's prices, tiers and overage rules

and issues the invoice. Send **aggregates with an idempotency key** per (tenant, meter, period), not millions of raw events, so a retried upload can't double-bill. Give customers a usage page with daily totals and the same dimensions you bill on (API key, model), so most disputes are settled before they reach support.

WORKED EXAMPLE: API CALLS AND TOKENS PER TENANT PER MONTH

Each request emits two events, `api_calls = 1` and `input/output_tokens` with the model name, both with IDs derived from the request ID. Kafka → Flink deduplicates by ID and writes daily totals per tenant, meter and model; Redis holds the running month for quota checks. On the 1st, monthly totals go to Stripe with idempotency keys. The dispute: the daily breakdown shows the customer's own count matches on most days, and the gap sits on three days when its client retried aggressively. Because the retries reused the request ID they were **not** billed; the remaining gap traces to a second API key the customer had forgotten, with per-key totals and sample request IDs to prove it.

COMMON MISTAKES

- Billing from a counter in the request path, with no events behind it.
- Generating a new event ID on each retry, so deduplication never matches.
- Aggregating by arrival time, so usage shifts between periods.
- Silently rewriting a closed month instead of issuing an adjustment.

7 Key terms

Usage event: immutable record of one billable action

Idempotency key: ID that makes a retry the same action

Soft / hard limit: warn vs block or overage

Meter: what is counted: calls, tokens, GB-hours

Event time: when usage happened, not when it arrived

Reconciliation: correcting fast counters from durable totals

8 Interview questions

Q1. Design usage metering for billing. BASIC

Emit immutable usage events with unique IDs, write them to a durable log, deduplicate and aggregate per tenant, meter and period, use a fast counter for live quota checks, and send period totals to the billing system.

Q2. Soft vs hard limits? BASIC

Soft limits warn the customer; hard limits block usage or switch it to overage pricing.

Q3. A customer disputes their bill. What do you need to answer? SCENARIO

Per-day, per-meter totals with the billed dimensions (API key, model), traceable to raw events and request IDs kept in the archive.

Q4. How do you enforce a monthly token quota without slowing every request? SCENARIO

Check and increment an in-memory counter in Redis per tenant and month, reserve an estimate before the model call, settle afterwards, and reconcile nightly with durable totals.

Q5. Kafka has exactly-once semantics. Why do you still need event IDs? TRICKY

Kafka's guarantees cover producer retries and processing inside Kafka; a client retrying the same request creates a new message that only an application-level ID can recognise.

Q6. Usage arrives after the month is invoiced. What do you do? TRICKY

Never rewrite the closed period; record it as an adjustment on the next invoice, with the original dates.

Q7. Why not bill straight from the Redis counter? TRICKY

It's approximate and has no history: it can lose or double increments and can't be recomputed or audited.

Next: SD-R-006 · Data Residency